Frederick Car Accessories

Client presentation and system handoff guide

Client presentation and system handoff guide

This document explains what the system does, how staff use it, how the frontend and backend are started, which database is used, and what each major folder and file is responsible for.

1. System summary

Frederick Car Accessories is a responsive web application for the shop at:

Door 5 #99 Cabaguio Avenue, Brgy. Gov Paciano Bangoy, Davao City

The application has two connected areas:

1. **Public storefront** - customers browse the shop services and submit a

service booking request.

2. **Inventory and booking portal** - authorized Admin and Employee users

manage inventory and process customer booking requests.

The application follows a client/server architecture:

```
Customer or staff browser
          |
          | HTTP requests
          v
React + Vite frontend
          |
          | /api requests
          v
Express backend
          |
          | mysql2 connection pool
          v
XAMPP MariaDB database
```

The frontend is not connected directly to MariaDB. The browser communicates with the Express backend, and only the backend communicates with the database. This protects database credentials and gives the server a central place to validate permissions and data.

2. Client-facing functionality

Public website

Customers can:

? View the Frederick Car Accessories storefront.

? Read the shop description, location, contact numbers, and email address.

? Browse the service catalog.

? Search services and filter them by category.

? Explore the interactive rotating car visualizer/video.

? Select a service from a card or car hotspot.

? Submit a booking request with:

? Full name

? Philippine mobile number

? Vehicle make and model

? Preferred service

? Appointment date and time

? Special notes or vehicle concerns

? Receive confirmation after the booking request is stored.

Staff portal

Staff open the portal at:

```
/inventory
```

After signing in, authorized staff can:

? View inventory products.

? Search by product name, SKU, or category.

? Filter by category and stock status.

? See total products, inventory value, low-stock items, and out-of-stock items.

? Add stock through the restock action.

? Export filtered or selected inventory as a CSV file.

? View customer booking requests.

? Update booking status.

? Choose a predefined response message for a booking.

Admin-only capabilities

Admins can additionally:

? Add products.

? Edit product details.

? Delete individual products.

? Delete selected products in bulk.

? Review employee account requests.

? Approve or reject employee account requests.

Employee capabilities

Employees can:

? View and search products.

? Restock products.

? Export inventory.

? View bookings.

? Approve, cancel, complete, or otherwise update bookings using the booking

workflow.

Employees cannot change the product catalog or approve employee account requests. These restrictions are enforced by the backend, not only hidden in the interface.

3. How a client or staff member uses the system

Customer booking flow

1. Open the public website.
2. Select **Book a service**, a service card, or a car hotspot.
3. Complete the booking form.
4. Review the entered details.
5. Submit the request.
6. Staff review the request from `/inventory`.
7. Staff update the status and select an appropriate response message.

Booking statuses are:

? `pending` - newly submitted and waiting for staff review.

? `confirmed` - the appointment is accepted.

? `completed` - the service has been completed.

? `cancelled` - the request cannot be accommodated.

Staff login flow

1. Open `/inventory`.
2. Enter the account email and password.
3. The server identifies whether the account is an Admin or Employee.
4. The browser receives an HTTP-only session cookie.
5. The portal loads the features allowed for that role.

There is no account-type selector on the login screen. A user's role is stored in the database and decided by the backend.

Employee account request flow

1. A prospective employee selects **Request employee access**.
2. They enter their name, email, and password.
3. The request is stored as `pending`.
4. An Admin opens **Account requests**.
5. The Admin approves or rejects the request.
6. Approval creates the active Employee account.
7. The employee can then sign in normally.

4. Database technology

The project uses **MariaDB**, provided locally by **XAMPP**. MariaDB is compatible with the MySQL protocol, so the project uses the `mysql2` Node.js package to connect to it.

Current local configuration:

Setting	Value
Database engine	MariaDB through XAMPP

Setting	Value
Host	`127.0.0.1`
Port	`3307`
Database	`frederick_autocare`
User	`root` for the current local setup
Password	Blank in the current local setup
API port	`3001`
Frontend port	Usually `5173`

For a production deployment, use a dedicated database user with a strong password instead of the local blank-password root account.

Database tables

The canonical schema is in database/schema.sql (./database/schema.sql).

Table	Purpose
`inventory_users`	Active Admin and Employee accounts
`inventory_sessions`	Server-side login sessions and expiry times
`inventory_account_requests`	Pending, approved, or rejected employee requests
`booking_requests`	Customer booking information and staff responses
`inventory_products`	Product names, SKUs, stock, thresholds, and prices

The database uses foreign keys so account and review relationships remain consistent. Product SKUs are unique, which prevents duplicate inventory identifiers.

Initial sample data

The schema includes 12 sample inventory products. The server also creates the initial local Admin and Employee accounts if they do not already exist:

Role	Email	Password
Admin	`admin@frederickauto.local`	`admin123`
Employee	`employee@frederickauto.local`	`employee123`

Change these values in `.env` before using the system outside local testing. Changing the environment values does not change an account that already exists in the database.

5. Initial setup and startup

Prerequisites

Install or have available:

? Node.js 20.19 or newer

? npm

? XAMPP with its MariaDB/MySQL module

Configure the project

From the project directory:

```
cd C:\Users\User\gihub-lerasan\sia\frederick-autocare
npm install
Copy-Item .env.example .env
```

Review `.env` and make sure `DB_PORT` matches the database port shown by XAMPP. The current project setup uses `3307`.

Initialize the database

The schema file creates the database, tables, indexes, constraints, and sample inventory. It can be imported through the local database administration tool, or executed with the XAMPP MariaDB client:

```
C:\xampp\mysql\bin\mysql.exe -h 127.0.0.1 -P 3307 -u root -p < database\schema.sql
```

When the local root password is blank, press Enter when prompted. Do not put a real production password in a command that may be saved in shell history.

Start the backend

Keep XAMPP MariaDB running, then open a terminal:

```
cd C:\Users\User\gihub-lerasan\sia\frederick-autocare
npm run server
```

The Express API should listen on:

```
http://127.0.0.1:3001
```

Verify the API and database connection:

```
Invoke-WebRequest http://127.0.0.1:3001/api/health
```

The response should report an `ok` status and a connected database.

Start the frontend

Open a second terminal:

```
cd C:\Users\User\gihub-lerasan\sia\frederick-autocare
npm run dev
```

Open the URL printed by Vite, normally:

```
http://localhost:5173
```

The Vite configuration proxies frontend `/api` requests to the Express API, so the browser can use relative paths such as `/api/products`.

Useful project commands

Command	Purpose
`npm run dev`	Start the Vite development frontend
`npm run server`	Start the Express backend
`npm run build`	Create the production frontend in `dist/`
`npm run preview`	Preview the production frontend build
`npm run lint`	Check JavaScript, JSX, and TypeScript lint rules

The frontend and backend are separate processes during development. Both must be running for the public booking and staff inventory workflows to work.

6. Code and folder structure

```
frederick-autocare/
■■■ public/
■■■ src/
■   ■■■ assets/
■   ■■■ components/
■   ■■■ data/
■   ■■■ hooks/
■   ■■■ App.jsx
■   ■■■ App.css
■   ■■■ index.css
■   ■■■ main.jsx
■■■ server/
■   ■■■ db.js
■   ■■■ index.js
■■■ database/
■   ■■■ schema.sql
■■■ .env
■■■ .env.example
■■■ eslint.config.js
■■■ index.html
■■■ package.json
■■■ package-lock.json
■■■ README.md
■■■ README_SYSTEM_GUIDE.md
■■■ vite.config.js
```

Root files

`package.json`

Defines the project name, scripts, and dependencies. Important dependencies include:

? React and React DOM for the interface.

? Vite for frontend development and bundling.

? Tailwind CSS for utility styling.

? Lucide React for interface icons.

? React Three Fiber, Drei, and Three.js for the 3D visualizer.

? Express for the API server.

? `mysql2` for MariaDB access.

? `dotenv` for environment configuration.

`package-lock.json`

Locks the exact dependency versions installed by npm. It keeps installs repeatable across machines.

`.env.example`

Documents the environment variables required by the backend. It is safe to share as a template because it contains no real secret. Copy it to `.env` for local use.

`.env`

Contains local database, API, CORS, account bootstrap, and session settings. It must not be committed or shared publicly.

`vite.config.js`

Configures the Vite React plugin and proxies `/api` requests to the Express backend during development.

`index.html`

The HTML entry document. Vite injects the React application into its root element.

`eslint.config.js`

Defines linting rules used by `npm run lint`.

Frontend entry files

`src/main.jsx`

The browser entry point. It mounts the React application into the HTML root element and loads the global stylesheet.

`src/App.jsx`

The top-level route switch:

- `/` renders the public `Storefront`.
- `/inventory` renders the staff `InventoryPortal`.

The `Storefront` composes the header, hero area, video/car visualizer, service catalog, booking modal, about section, contact information, and footer.

`src/App.css`

Contains application-specific layout and visual styles, including hero, storefront sections, buttons, cards, booking presentation, and responsive behavior.

`src/index.css`

Contains global CSS rules, Tailwind imports, base typography, shared backgrounds, and global utility styles.

`src/components/`

Each component owns one reusable area of the interface.

`Header.jsx`

Displays the storefront navigation and the main booking call-to-action.

`Footer.jsx`

Displays shop branding, contact information, and footer navigation content.

`ServiceCatalog.jsx`

Displays the service list with search and category filtering. It passes the chosen service to the booking flow.

`ServiceCard.jsx`

Renders one service card, including its title, category, description, price estimate, and booking button.

`BookingModal.jsx`

Implements the controlled multi-step booking form. It validates customer inputs, displays a review step, submits to `POST /api/bookings`, and shows a success or error state.

`CarVisualizer.jsx`

Builds the optional Three.js sedan visualizer from reusable mesh pieces. It uses React Three Fiber and Drei for the canvas, camera controls, lighting, hotspots, and 3D interaction.

`CarVideoScrubber.jsx`

Provides the current rotating-car video experience. It supports mouse and touch dragging, autoplay, reset, service hotspots, and browser-local hotspot calibration. It also displays a fallback when the video cannot load.

`InventoryPortal.tsx`

Contains the staff portal and its major workflows:

- Login and employee registration request forms.
- Session restoration and logout.
- Role-aware navigation.
- Product loading, filtering, pagination, selection, and CRUD actions.

? Restocking.

? Inventory CSV export.

? Account-request approval for Admins.

? Booking listing and status management.

? Predefined booking response messages.

This is the largest frontend file because the inventory portal is currently implemented as one feature module with internal components and typed data models.

`src/data/`

`servicesData.js`

Stores the public service catalog used by service cards, booking dropdowns, and visualizer/video service hotspots. Keeping the catalog in one module prevents different parts of the storefront from displaying inconsistent service information.

`src/hooks/`

`useBookingState.js`

Encapsulates the open/close state for the booking modal so the header, service catalog, video hotspots, and modal can work together without duplicating modal state logic.

`src/assets/`

Contains imported frontend assets such as the hero image and starter SVG assets.

`public/`

Contains files served directly without being imported into the JavaScript bundle:

? `car-360.mp4` - rotating vehicle video.

? `favicon.svg` - browser tab icon.

? `icons.svg` - public SVG resource.

Backend files

`server/db.js`

Loads database configuration from environment variables and creates a reusable `mysql2/promise` connection pool. The rest of the backend imports this pool instead of opening a new database connection for every request.

`server/index.js`

The Express application. It:

? Parses JSON requests.

? Configures CORS and cookies.

? Creates or migrates required authentication and booking columns.

? Seeds default accounts when needed.

? Hashes and verifies passwords with scrypt.

? Creates and validates HTTP-only sessions.

? Applies Admin and Employee authorization checks.

? Validates product and booking input.

? Exposes authentication, account-request, booking, product, restock, and

health endpoints.

? Performs graceful database pool shutdown.

Database files

`database/schema.sql`

The canonical MariaDB schema. It creates the `frederick_autocare` database, all application tables, indexes, foreign keys, and starter inventory rows.

Generated and local-only folders

`node_modules/`

Installed npm dependencies. It is generated by `npm install` and should not be committed.

`dist/`

The production frontend output generated by `npm run build`.

7. API and data flow

Public booking

```
BookingModal.jsx
  -> POST /api/bookings
  -> server/index.js validates request
  -> booking_requests table receives row
  -> success response returns to BookingModal
```

Staff inventory

```
InventoryPortal.tsx
  -> GET /api/products
  -> server/index.js checks session
  -> db.js queries inventory_products
  -> JSON response renders the dashboard
```

Product creation, editing, restocking, deleting, and bulk deleting follow the same pattern. The backend checks the authenticated role before changing data.

Staff authentication

```
Login form
  -> POST /api/auth/login
  -> scrypt password verification
  -> inventory_sessions row created
  -> HTTP-only cookie returned
  -> later requests include the cookie automatically
```

The system does not currently use an API key. The session cookie is intentionally HTTP-only, so normal frontend JavaScript cannot read it.

Main API endpoints

Method	Endpoint	Access	Purpose
GET	/api/health	Public	Check API and database health
POST	/api/bookings	Public	Create a customer booking
POST	/api/auth/login	Public	Start a staff session
POST	/api/auth/register	Public	Request Employee access
GET	/api/auth/me	Signed in	Restore current session
POST	/api/auth/logout	Signed in	End current session
GET	/api/auth/requests	Admin	List account requests
POST	/api/auth/requests/:id/approve	Admin	Approve an Employee
POST	/api/auth/requests/:id/reject	Admin	Reject an Employee request
GET	/api/bookings	Admin/Employee	List booking requests
PUT	/api/bookings/:id/status	Admin/Employee	Update booking status
GET	/api/products	Signed in	List inventory
POST	/api/products	Admin	Add a product
PUT	/api/products/:id	Admin	Edit a product
POST	/api/products/:id/restock	Admin/Employee	Add stock
DELETE	/api/products/:id	Admin	Delete a product
POST	/api/products/bulk-delete	Admin	Delete selected products

8. Inventory export

The inventory portal exports the current inventory view to CSV.

? The top **Export** button exports products matching the current search and filters.

? **Export selected** exports the products selected in the table.

? The CSV includes product, SKU, category, stock, threshold, stock status, unit price, and calculated inventory value.

? Files include the export scope and date in their filename.

? Values are escaped for commas and quotation marks.

? Spreadsheet formula-like values are prefixed to reduce formula injection risk.

? An empty search/filter result displays a message instead of downloading an empty file.

CSV is supported by Microsoft Excel, Google Sheets, LibreOffice Calc, and other spreadsheet applications.

9. Maintenance and troubleshooting

API cannot connect to MariaDB

Check:

1. XAMPP MariaDB is running.
2. `.env` uses the same `DB_PORT` shown by XAMPP.
3. `DB_HOST`, `DB_USER`, and `DB_NAME` are present.
4. No separate MySQL service is already using the configured port.
5. `http://127.0.0.1:3001/api/health` reports a database connection.

The browser says `Cannot POST /api/...`

Make sure the current backend is running with:

```
npm run server
```

Then restart Vite if its proxy configuration was changed.

Login credentials appear incorrect

Default account environment values are only used when the account is first created. Existing database rows keep their existing password hash. Changing `.env` alone does not reset an already-created account.

The inventory is empty

Confirm that:

1. The API health endpoint is connected.
2. The `inventory_products` table exists.
3. The schema was imported into `frederick_autocare`.
4. The browser is using the current Vite and Express processes.

A port is already in use

Stop the old development/API process, then start the correct project process again. Only one service should own the configured API port, and only one database service should own the configured MariaDB port.

Validation before handoff

Run:

```
npm run lint
npm run build
```

Both commands should pass before presenting a new build or deploying it.

10. Security and deployment notes

- ? Never commit `.env`.
- ? Never expose database credentials in `VITE_*` variables.
- ? Replace the sample Admin and Employee passwords before real use.
- ? Use a dedicated MariaDB account with a strong password in production.
- ? Keep the database private and expose only the API to the intended network.
- ? Use HTTPS when the application is deployed outside a local machine.
- ? The session cookie is HTTP-only so it cannot be read by ordinary frontend

scripts.

? Backend authorization remains the source of truth for Admin and Employee

permissions.

? Keep backups of the MariaDB database before schema changes or deployment.

11. Short presentation script

Frederick Car Accessories uses a responsive React website for customers and a secured staff portal for daily operations. Customers can discover services and submit booking requests online. Staff members sign in to manage inventory, restock products, export CSV reports, and process bookings. Administrators have additional controls for product management and employee account approval.

The frontend is powered by React and Vite. The backend is an Express API. The system stores persistent users, sessions, bookings, and inventory in MariaDB running through XAMPP. The backend is the only layer that connects to the database, validates input, and applies role permissions. This separation keeps the user interface responsive while protecting database access and centralizing business rules.